

HealthAI

7-Step ML Learning Tool

Project Report | SENG 430 | Junior Engineers | May 2026

Name	Role
Göksu Bulut	Product Owner
Kerim Efe Kocacık	UX Designer
Ceren Gökdere	Lead Developer
Elif İstanbulluoğlu	Scrum Master / Developer
Emre Tek	QA / Documentation Lead

1 Project Overview

HealthAI is a browser-based interactive learning tool designed to help healthcare professionals — doctors, nurses, clinical researchers, and students — understand machine learning in real clinical contexts. Users select one of 20 medical specialties and walk through a guided 7-step pipeline, from data exploration to ethics and bias auditing, with no installation required. The project was developed over five sprints by a team of five, culminating in a jury presentation on May 6, 2026.

2 Architecture

The system follows a clean client-server separation. The **frontend** is built in plain HTML, CSS, and JavaScript — a deliberate choice that keeps the tool accessible in any browser without framework overhead. The **backend** is a FastAPI (Python) application that handles model training, prediction, and metric computation via scikit-learn. Communication between layers is REST-based, with lightweight JSON payloads. User session data is stored entirely in **browser Session Storage**, eliminating the need for a database. The entire application is containerised with **Docker**, ensuring consistent environments across development machines and making deployment reproducible regardless of local setup.

The ML engine supports six classification models across 20 clinical domains, each with domain-specific datasets and clinical display names. Step 6 integrates SHAP (KernelExplainer) for model explainability, generating feature importance charts and patient-level waterfall plots. Step 7 implements subgroup fairness analysis and an interactive EU AI Act compliance checklist, with a PDF certificate export endpoint.

3 Key Decisions

Plain HTML/CSS/JS over a frontend framework. Not using React or Vue kept the barrier to entry low for both users and contributors. No build step, runs anywhere, and the codebase remained approachable for a cross-disciplinary team.

FastAPI over Flask. FastAPI's automatic request validation, async support, and auto-generated OpenAPI docs made it a better fit for an ML-serving backend. It also kept frontend-backend integration in sync across sprints without manual documentation overhead.

Docker for containerisation. Wrapping the app in Docker containers eliminated environment inconsistencies across team members' machines. Onboarding was faster and the setup provided a clean deployment path from the start.

Browser Session Storage over a backend database. Storing state client-side eliminated infrastructure complexity. The trade-off — sessions do not persist across page refreshes — is acceptable for a learning tool but would be a blocker for a production clinical system.

SHAP KernelExplainer over model-specific explainers. A model-agnostic explainer works consistently across all six supported models without separate implementations, at the cost of computational speed and approximation accuracy. A disclaimer was added noting that SHAP values are approximations, not exact attributions.

4 Challenges

Dataset ingestion and processing logic. Building a unified pipeline that correctly reads, validates, and processes CSV datasets across 20 different clinical domains was the most demanding engineering challenge. Each domain had different column structures, missing value patterns, and normalisation requirements. Designing a generalised ingestion layer without hardcoding domain-specific logic required significant iteration.

SHAP performance on larger datasets. KernelExplainer's computational cost became noticeable on datasets with more features and larger sample sizes. The team mitigated this with background dataset sampling but did not fully resolve the latency issue within the sprint timeline.

Reducing model training time. As the number of supported models and domains grew, training times during user sessions became a bottleneck. The team experimented with dataset subsampling and parameter constraints to bring training latency to an acceptable level, balancing speed against model quality.

Clinical language vs. database column names. Step 6 initially displayed raw column names in feature importance charts such as `bp_sys`, `gluc_fasting`. Mapping all columns to clinical display names across 20 domains required significant domain research and cross-team coordination, completed mid-Sprint 4.

5 Lessons Learned

Technical decision-making under uncertainty. Choosing between frameworks, explainability libraries, and data handling strategies — often without a clear right answer — developed the team's ability to evaluate trade-offs systematically rather than defaulting to the most familiar option.

Managing a large-scale project across sprints. Coordinating five roles over five sprints with real deadlines taught the team how scope, priorities, and dependencies shift over time, and how important early planning decisions are to later sprint velocity.

Manual and automated testing every sprint. Committing to both manual functional tests and automated checks in each sprint — rather than saving QA for the end — meant bugs were caught close to when they were introduced, making the codebase significantly more stable by Sprint 5.

Regular reporting as a discipline. Weekly progress reports, sprint retrospectives, and ISO 42001 compliance documentation created a habit of articulating what was built, what broke, and what changed — keeping the whole team aligned across five sprints.

Working across a diverse technology stack. The project required simultaneous work with HTML/CSS/JS, Python, FastAPI, scikit-learn, SHAP, and Docker. Managing this breadth — keeping each layer functional while integrating them — was a real engineering challenge and a significant learning experience.

Building for accessibility. Designing for healthcare professionals with varying technical backgrounds pushed the team to think carefully about language, layout, and interaction design. Clinical display names, plain-language bias explanations, and the guided step-by-step format all emerged from asking: would a nurse with no ML background understand this?